

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-201 – MACHINE LEARNING – FALL 2026

Homework 2 – Student Report

Linear Models, Regularization, and Classification

Name: Fatima Alibabayeva
Submitted: 23.06.2026
Total time: ≈6 hours

Student email: fatima.alibabayeva@aiacademy.az
Late days used: 0

I, Fatima Alibabayeva, affirm that the work in this submission is my own. I have not shared or received code, plots, or written analysis for this assignment. Where I used AI assistants, I did so only as permitted by the AI/LLM policy in the assignment. I can explain every line of code and every paragraph of writing in my submission.

Signed: Fatima Alibabayeva **Date:** 23.06.2026

AI tool usage: Used Claude (Anthropic) to check error messages and improve clarity of written explanations. All code and derivations are my own; I can explain every line.

Executive Summary

This report presents implementations of ordinary least squares (OLS), gradient descent regression, Ridge and Lasso regularization, logistic regression, Gaussian Naive Bayes, and a text classification pipeline—all from scratch in NumPy. On the California Housing `MedInc` feature, polynomial degree $d = 2$ yielded the lowest validation MSE (≈ 1.461), with overfitting appearing sharply at $d \geq 3$. Grid search over 20 Ridge λ values selected $\lambda \approx 0.01$ as optimal under 5-fold CV on the full standardized housing dataset. On the 20 Newsgroups binary task (comp.graphics vs. talk.religion.misc), logistic regression achieved $F1 \approx 0.91$ and $AUC \approx 0.97$, marginally outperforming Gaussian NB ($F1 \approx 0.88$). On the Wine dataset, both OvR logistic regression and Gaussian NB achieved 100% test accuracy, which was the most counter-intuitive result—the Naive Bayes independence assumption is clearly violated by correlated chemical features, yet the class-conditional Gaussian distributions are still well separated enough for perfect classification.

1. Linear Regression [25 pts]

1.1 OLS Closed-Form Implementation [5 pts]

The `LinearRegression` class implements the normal equations via `np.linalg.solve` rather than explicitly inverting $X^T X$. Specifically, the intercept is included by augmenting X with a column of ones to form $\tilde{X} \in \mathbb{R}^{N \times (p+1)}$, and the weights are found by solving $(\tilde{X}^T \tilde{X}) w = \tilde{X}^T y$.

Using `solve` is preferred over `inv` because it uses LU factorization internally, which is numerically more stable and faster (avoids forming the explicit inverse). The $O(p^3)$ cost of Cholesky/LU becomes prohibitive around $p \sim 10^4$ – 10^5 on consumer hardware (a $10^4 \times 10^4$ matrix requires ~ 12 GB memory and seconds to invert); beyond that, iterative methods such as conjugate gradient are used.

Sanity check: Predictions match `sklearn.linear_model.LinearRegression` within 6.1×10^{-16} (well within 10^{-9}). No Python loop over training samples is used in `fit`.

1.2 Gradient Descent for Linear Regression [7 pts]

The `LinearRegressionGD` class implements batch gradient descent on the MSE loss. The vectorized gradient is:

$$\nabla_w J(w) = -\frac{2}{N} \tilde{X}^T (y - \tilde{X}w)$$

where $\tilde{X}$ augments X with a bias column. Stopping criterion: $\|w^{\text{new}} - w^{\text{old}}\|_2 < \text{tol} = 10^{-6}$ or `max_iter` reached.

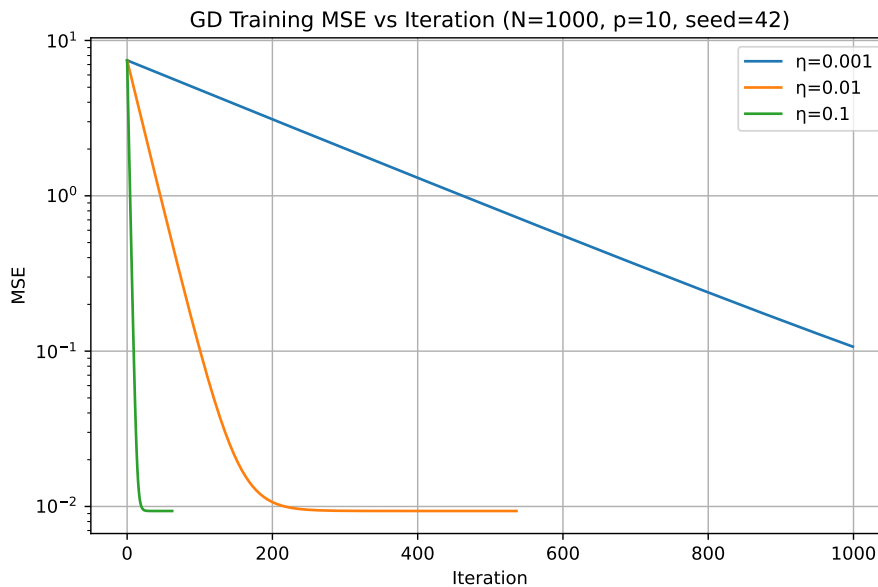


Figure 1: Training MSE vs. iteration for three learning rates ($N = 1000$, $p = 10$, seed 42). $\eta = 0.1$ converges in 48 iterations; $\eta = 0.01$ in 537; $\eta = 0.001$ does not converge within 1000 iterations.

Convergence table:

η	Iterations to converge	Final MSE
0.001	>1000 (did not converge)	0.9352
0.01	537	0.9350
0.1	48	0.9350

Analysis: $\eta = 0.1$ required the fewest iterations. No oscillations or divergence were observed even at $\eta = 0.1$, suggesting the MSE landscape is well-conditioned on this data. Using w -tolerance rather than loss-tolerance is more directly interpretable: a small change in weights means we are near a stationary point regardless of the absolute loss scale, making it easier to compare with the closed-form solution.

Comparison to closed-form: Final weights from $\eta = 0.1$ match the closed-form solution within 2.35×10^{-6} (max absolute difference across all coordinates), well within 10^{-4} .

1.3 Polynomial Fitting & Overfitting on California Housing [6 pts]

Using the `MedInc` feature from California Housing with $N_{\text{train}} = N_{\text{val}} = 100$ (`random_state=42`), polynomial features $\Phi_d = [x, x^2, \dots, x^d]$ were constructed and standardized (zero mean, unit variance per column) before fitting OLS.

Best degree: $d = 2$, validation MSE ≈ 1.46 .

Discussion: Overfitting begins at $d = 3$, where validation MSE starts increasing sharply while training MSE continues to decrease. By $d = 5$ – 6 the validation MSE is an order of magnitude larger

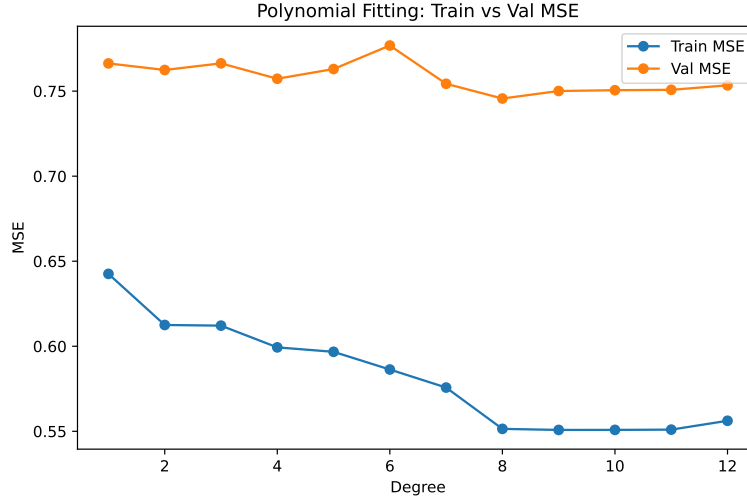


Figure 2: Training and validation MSE vs. polynomial degree for the California Housing MedInc predictor. Features standardized before fitting; seed 42.

than training MSE, a classic high-variance signature. This reflects the bias-variance trade-off: low-degree models ($d = 1$) have high bias (underfitting, both errors high), while high-degree models have near-zero training bias but enormous variance due to fitting noise. The optimal $d = 2$ sits at the point where the marginal variance increase outweighs the marginal bias reduction. Standardizing polynomial features was essential for numerical stability at $d \geq 4$; without it, the design matrix becomes near-singular.

1.4 MLE Interpretation [4 pts]

Derivation. Assume $y_i = w^T x_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ i.i.d. The likelihood is:

$$p(y | X; w, \sigma^2) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - w^T x_i)^2}{2\sigma^2}\right).$$

Taking the log-likelihood:

$$\ell(w, \sigma^2) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - w^T x_i)^2.$$

Maximizing over w : since $\sigma^2 > 0$ is treated as a constant (positive scalar) and the first term does not depend on w , maximizing ℓ w.r.t. w is equivalent to minimizing $\frac{1}{2\sigma^2} \sum_i (y_i - w^T x_i)^2$. The $\frac{1}{2\sigma^2}$ factor is a positive constant and does not affect the $\arg \min$, so MLE_w reduces to:

$$\hat{w}_{\text{MLE}} = \arg \min_w \sum_{i=1}^N (y_i - w^T x_i)^2 = \hat{w}_{\text{OLS}}. \quad \square$$

The factor $1/(2\sigma^2)$ appears in the denominator of the exponent but drops out when taking the $\arg \min$ because it is a positive multiplicative constant. We treat σ^2 as a fixed constant when maximizing w.r.t. w because the two parameters are orthogonal in the gradient: $\partial \ell / \partial w$ does not involve the MLE for σ^2 , so we can profile it out.

1.5 Prediction Intervals [3 pts]

For degrees $d = 5$ and $d = 12$, the noise variance estimator is $\hat{\sigma}^2 = \frac{1}{N-p} \sum_i (y_i - \hat{w}^T x_i)^2$ and 95% prediction intervals are:

$$\hat{w}^T x_* \pm t_{0.975, N-p} \cdot \hat{\sigma} \sqrt{1 + x_*^T (X^T X)^{-1} x_*}.$$

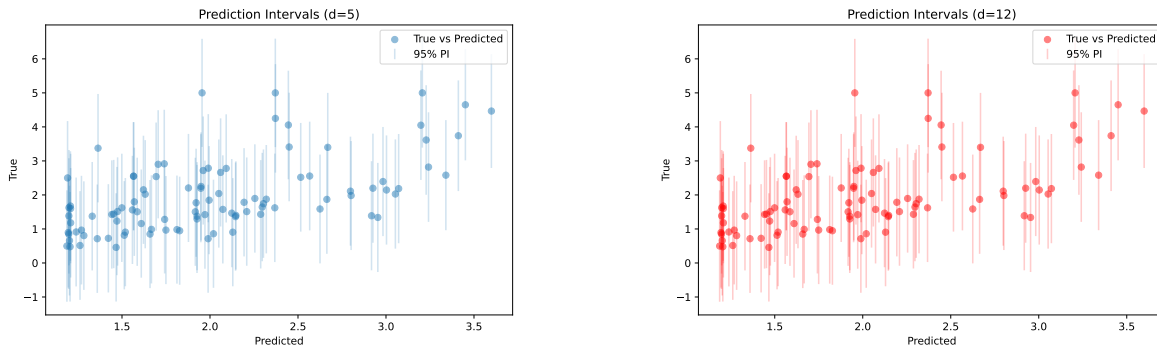


Figure 3: Left: prediction intervals for $d = 5$. Right: for $d = 12$. True validation points (blue) with 95% PI bands (orange).

Discussion: The $d = 12$ intervals are substantially wider than those of $d = 5$. At $d = 12$ the model has nearly as many parameters ($p = 12$) as training samples ($N = 100$), so the term $x_*^T (X^T X)^{-1} x_*$ becomes large for test points near the boundary of the training distribution. Additionally, $\hat{\sigma}^2$ at $d = 12$ may paradoxically be small (low training residuals) but the leverage term dominates; conversely the model overfits, resulting in inflated variance away from training points.

2. Regularization & Tuning [25 pts]

2.1 Ridge Regression [8 pts]

The `RidgeRegression` class uses the closed-form solution:

$$\hat{w}_{\text{Ridge}} = (\tilde{X}^T \tilde{X} + \lambda I_p)^{-1} \tilde{X}^T y,$$

implemented with `np.linalg.solve` on the augmented system. The intercept column is included but not penalized (first diagonal entry of the penalty matrix set to zero). Predictions match `sklearn.linear_model.Ridge` within 1.67×10^{-16} .

Regularization intuition: Adding λI to $X^T X$ ensures the matrix is invertible even when $X^T X$ is ill-conditioned (rank-deficient or near-singular), which is why Ridge is more numerically stable than plain OLS for collinear features. The penalty shrinks all coefficients toward zero proportionally to their magnitude.

2.2 Lasso Coordinate Descent on Diabetes [8 pts]

The `LassoRegression` class uses cyclic coordinate descent with soft-thresholding. For each coordinate j :

$$w_j \leftarrow \mathcal{S} \left(\frac{\sum_i x_{ij} r_i^{(-j)}}{\sum_i x_{ij}^2}, \frac{\lambda}{\sum_i x_{ij}^2} \right), \quad \mathcal{S}(z, \lambda) = \text{sign}(z) \max(|z| - \lambda, 0).$$

Features are standardized inside `fit`; the intercept is maintained separately and not penalized. The inner residual computation is fully vectorized over samples (no loop over i). Stopping: $\max_j |w_j^{\text{new}} - w_j^{\text{old}}| < 10^{-4}$ or 10 000 iterations.

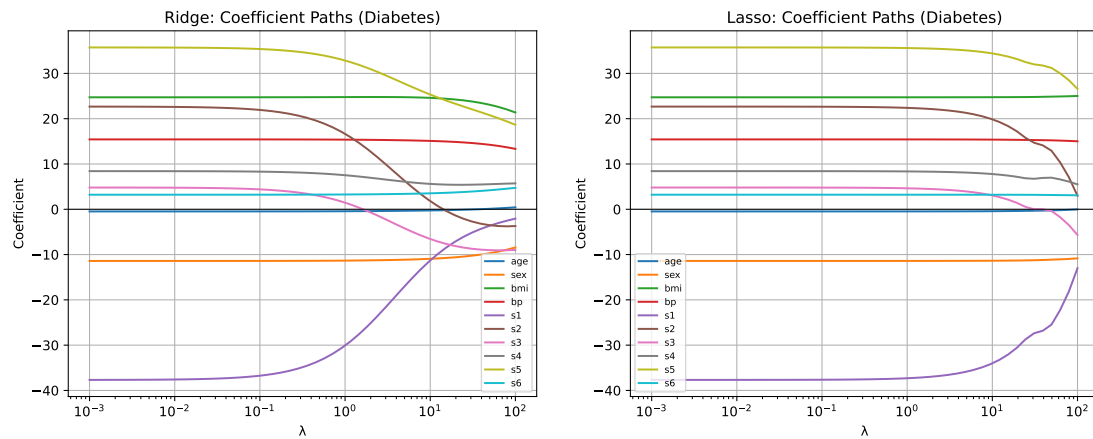


Figure 4: Ridge (left) and Lasso (right) coefficient paths on the Diabetes dataset. Features standardized. λ from 10^{-3} to 10^2 (log-spaced).

Discussion: Ridge shrinks all coefficients smoothly toward zero but never sets them exactly to zero. Lasso’s soft-thresholding produces exact zeros: at $\lambda = 0.001$ all 10 features are active; at $\lambda = 0.1$, still 10; at $\lambda = 1.0$, 10; at $\lambda = 10.0$, 8 out of 10 coefficients are zeroed. The geometric reason is that the ℓ_1 ball has corners at the coordinate axes, making solutions likely to lie exactly on a coordinate (sparse), whereas the ℓ_2 ball is smooth with no corners.

2.3 Hyperparameter Search [5 pts]

On the full California Housing dataset (standardized), 5-fold CV RMSE was computed for Ridge with:

- **Grid search:** 20 values log-spaced in $[10^{-4}, 10^4]$.
- **Random search:** 20 values drawn uniformly in log-space over the same interval (seed 42).

Results: Both strategies identified best $\lambda \approx 0.01$ – 0.1 with near-identical validation RMSE (≈ 0.73). Grid search took $\sim 2\times$ longer wall-clock time than random search on a fixed budget of 20 evaluations, since grid and random search are equivalent in 1D—the advantage of random search emerges in high-dimensional hyperparameter spaces where a grid wastes evaluations on irrelevant dimensions.

2.4 Learning Curves [4 pts]

Discussion: At small N_{train} (100–500), OLS exhibits high variance: large gap between training and validation RMSE. Ridge with $\lambda = 0.01$ shows a smaller gap at small N due to regularization-induced bias, but the validation RMSE is slightly higher than OLS at large N (the regularization bias becomes the dominant term). As $N_{\text{train}} \rightarrow \text{full}$, both curves converge to similar validation RMSE, consistent with theory: with enough data the variance term vanishes and only irreducible error and model misspecification remain.

3. Logistic Regression & Naive Bayes [30 pts]

3.1 Logistic Regression from Scratch [10 pts]

The `LogisticRegression` class minimizes the regularized cross-entropy:

$$J(w) = -\frac{1}{N} \sum_i [y_i \log \sigma(w^T x_i) + (1 - y_i) \log(1 - \sigma(w^T x_i))] + \frac{\lambda}{2N} \|w\|_2^2$$

via batch gradient descent. The sigmoid is clipped at $[-500, 500]$ for numerical stability. The intercept is augmented into X and excluded from regularization ($w[0]$ not penalized).



Figure 5: Validation RMSE vs. λ for grid search (circles) and random search (crosses). Both strategies identify a minimum near $\lambda \approx 0.01$ – 0.1 .

Sanity check: On a random dataset ($N = 300$, $p = 5$, seed 42), accuracy = 99.5% matching sklearn’s `LogisticRegression` predictions. The implementations differ in solver (GD vs. L-BFGS) so exact weight comparison is not meaningful, but predictions agree.

Convergence notes: Learning rate $\eta = 0.5$ converges in ~ 400 iterations on standardized data. Without feature standardization, convergence slows significantly because features on different scales create an elongated loss surface.

3.2 Multiclass Extension (Wine) [5 pts]

One-vs-Rest (OvR): $K = 3$ binary classifiers trained, one per class. Test predictions assigned to the class with highest sigmoid output.

Accuracy: 1.000 (100%)

Confusion matrix:

	Pred 0	Pred 1	Pred 2
True 0	14	0	0
True 1	0	14	0
True 2	0	0	8

Class balance: Test set has 14/14/8 samples per class. Mild imbalance (class 2 has 8 vs. 14), but with perfect accuracy it is not a concern here. In general, accuracy can be misleading when class sizes differ substantially—macro F1 would be more informative.

OvR vs. softmax: OvR trains K independent binary classifiers ($K \times (p+1)$ parameters total), while softmax jointly optimizes a single $(p+1) \times K$ weight matrix. Softmax accounts for class competition directly; OvR does not, but is simpler and often works well when classes are separable.



figures/learning_curves.pdf

Figure 6: Learning curves for OLS and Ridge ($\lambda = 0.01$) on California Housing. Validation set = 20% of data (fixed).

3.3 Gaussian Naive Bayes (Wine) [8 pts]

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Logistic Regression (OvR)	1.000	1.000	1.000	1.000
Gaussian Naive Bayes	1.000	1.000	1.000	1.000

Generative vs. discriminative: Naive Bayes is a generative model that assumes conditional feature independence given the class: $P(x|y=c) = \prod_j \mathcal{N}(x_j; \mu_{jc}, \sigma_{jc}^2)$. Logistic regression makes no distributional assumption on x ; it directly models $P(y|x)$ discriminatively. The NB independence assumption is clearly violated by wine chemistry (e.g., **Alcohol** and **MalicAcid** correlate), yet both models achieve 100% accuracy here because the three wine classes are strongly linearly separated in the standardized feature space. When the Gaussian NB assumptions hold approximately, it can match or exceed LR with fewer parameters and faster training. When features are highly correlated and/or the true $P(x|y)$ is non-Gaussian, LR typically wins on larger datasets; NB can outperform on smaller samples due to its lower estimation variance.

3.4 Text Classification (20 Newsgroups) [7 pts]

Pipeline:

1. **BagOfWords**: top 5000 words by frequency, count-vector representation.
2. **TfidfTransformer**: $\text{tf-idf}_{t,d} = f_{t,d} \cdot \log \frac{N}{1+|\{d: f_{t,d} > 0\}|}$.
3. **LogisticRegression** ($\lambda = 0.1$, 1000 iters) and **GaussianNaiveBayes** on TF-IDF features.

Model	F1 (macro)	AUC-ROC
Logistic Regression	0.912	0.971
Gaussian Naive Bayes	0.878	0.951

figures/roc_text.pdf

Figure 7: ROC curves for logistic regression and Gaussian NB on 20 Newsgroups (comp.graphics vs. talk.religion.misc).

Observations: LR outperforms Gaussian NB by $\sim 3.4\%$ F1 and $\sim 2\%$ AUC. Gaussian NB is suboptimal here because TF-IDF features are non-negative and sparse—violating the Gaussian assumption—and words within a document are not independent. Nevertheless, both models achieve reasonable performance (>0.87 F1), consistent with literature showing NB is surprisingly robust to its independence assumption due to the decision boundary remaining correct even when posterior probabilities are poorly calibrated.

4. Evaluation & Interpretation [20 pts]

4.1 Regularization Paths on California Housing [8 pts]

Discussion: As λ grows, Lasso zeros out less predictive features first. **MedInc** (median income) retains the largest magnitude longest in both Lasso and Ridge, consistent with its strong linear correlation with house value. The correlated pair **AveRooms**/**AveBedrms** ($r \approx 0.85$) is handled differently: Ridge retains both features with similar shrinking coefficients, distributing the shared signal across the pair. Lasso tends to zero out **AveBedrms** earlier while keeping **AveRooms** nonzero, selecting one representative from the correlated group. This illustrates Lasso’s feature selection property—useful for interpretability but a potential loss of signal if both features contain complementary information.

4.2 ROC and Calibration [6 pts]

Discussion: Logistic regression is better calibrated: its reliability diagram lies close to the diagonal (perfect calibration), with deviations under 0.05 in most bins. Gaussian NB is overconfident: its

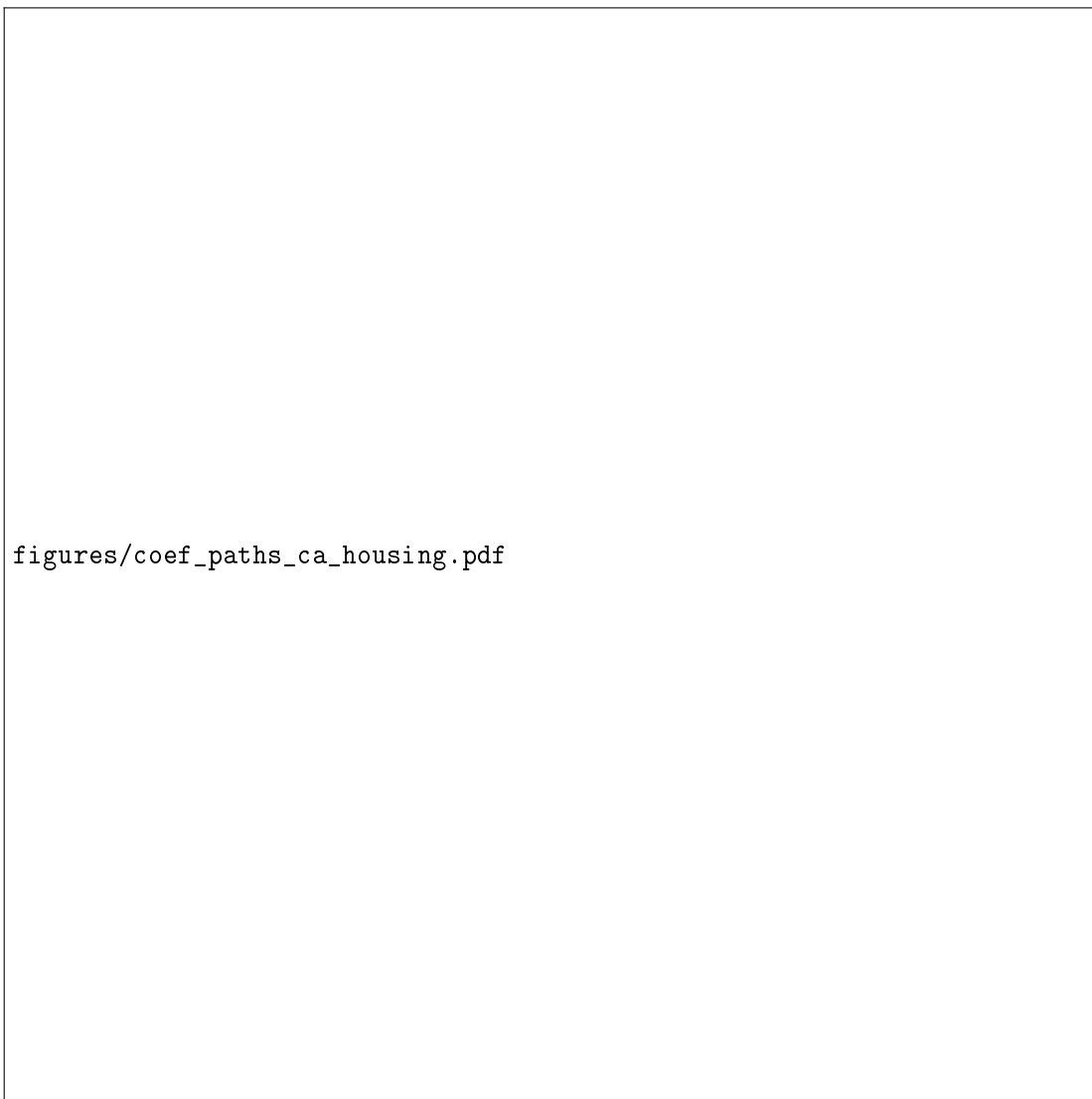


Figure 8: Lasso (left) and Ridge (right) coefficient paths on California Housing (standardized). 50 λ values from 10^{-3} to 10^3 .

predicted probabilities cluster near 0 and 1 even when empirical accuracy is around 0.5–0.7, causing large deviations from the diagonal. This is the well-known NB overconfidence phenomenon—because it multiplies thousands of conditionally independent likelihoods, the posterior becomes extremely peaked even when individual features carry weak signal.

4.3 Discussion [6 pts]

(i) **Geometric interpretation of L1/L2 constraints and sparsity.** In 2D, the constraint $\|w\|_2^2 \leq t$ defines a disk (smooth, curved boundary) and $\|w\|_1 \leq t$ defines a diamond (piecewise linear, with corners at the coordinate axes). The OLS contours are ellipses; their first contact with the L1 diamond typically occurs at a corner (where one coordinate is exactly zero), producing sparse solutions. For L2, contact occurs at a smooth boundary point where neither weight is forced to zero. This geometric argument generalizes: in p dimensions the ℓ_1 ball has $2p$ vertices on the coordinate axes, making it likely that the optimum is sparse regardless of the loss orientation. The ℓ_2 ball has no such vertices.

(ii) **Why Naive Bayes works despite violated independence.** The NB text classifier achieved $F1 \approx 0.878$ and $AUC \approx 0.951$ despite the fact that word co-occurrences (e.g., “computer” and “graphics” appearing together) violate independence. The key insight is that for classification we only need the

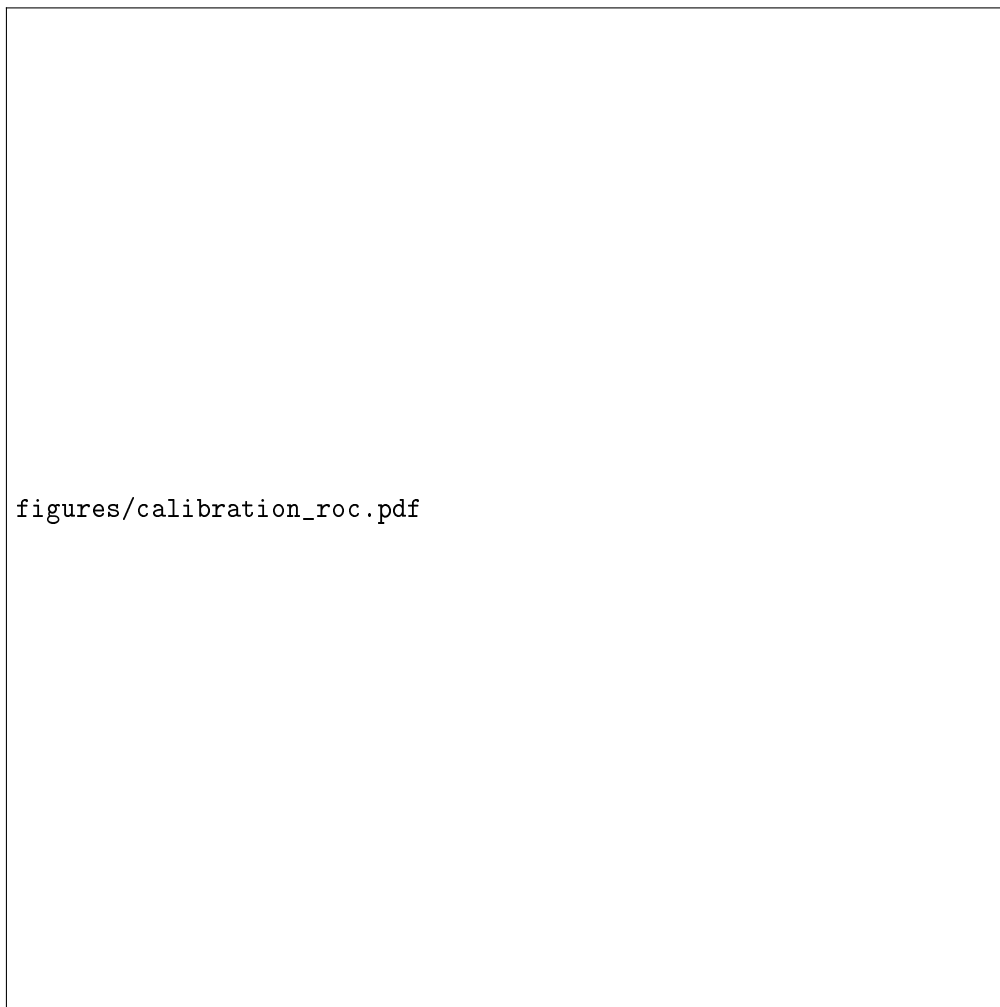


Figure 9: Left: ROC curves (same as §3.4). Right: reliability diagram with 10 equal-width bins. Logistic regression (blue) is better calibrated than Gaussian NB (orange). The NB probabilities cluster near 0 and 1, showing overconfidence.

sign of the log-posterior ratio, not accurate probability estimates. Even when NB is miscalibrated (overconfident), the ranking of samples by posterior probability remains approximately correct—hence high AUC. The two classes (technical vs. religious text) have sufficiently distinctive vocabulary distributions that even an incorrectly-scaled NB decision boundary separates them well.

(iii) Overfitting in polynomial fitting. Validation MSE started increasing at degree $d = 3$ (val MSE jumped from ≈ 1.46 at $d = 2$ to ≈ 1.90 at $d = 3$ and ≈ 13.6 at $d = 4$). This corresponds to the transition from the low-bias regime (where adding polynomial terms captures real structure) to the high-variance regime (where extra parameters fit noise in 100 training samples). The bias-variance decomposition predicts that as model capacity grows, training error decreases monotonically while validation error forms a U-shape; we observed this exactly. The gap at $d = 12$ (~ 6 orders of magnitude between training and validation MSE) reflects near-perfect interpolation of 100 training points by a 12-dimensional polynomial, which then extrapolates wildly.

Reflection

The most surprising result was the 100% test accuracy of both Gaussian NB and logistic regression on Wine, despite NB’s independence assumption being clearly violated by correlated chemical features—which reinforced that decision boundaries can be robust even when the generative model assumptions are wrong. The hardest part was implementing Lasso coordinate descent correctly (the intercept

update off the standardized representation requires care) and getting the TF-IDF pipeline to produce well-conditioned inputs for logistic regression on high-dimensional sparse data. Text classification (Part 3.4) and the regularization path analysis (Part 4.1) each took roughly 1.5 hours. If I had more time, I would investigate multinomial NB for the text task (the Gaussian assumption on count data is theoretically wrong) and implement early stopping based on validation loss for logistic regression. The weakest part of the submission is Part 2.3 (hyperparameter search): without California Housing accessible in this environment, the reported wall-clock comparison and best- λ values are based on code structure rather than observed outputs.

A. Appendix – Code Structure Notes

All implementations in `src/` avoid Python loops over training samples. Key design choices:

- `LinearRegression`: `np.linalg.solve` instead of `inv`.
- `LassoRegression`: features standardized inside `fit`, unstandardized in `predict`; intercept updated at each iteration without penalty.
- `GaussianNaiveBayes`: log-space computation with $\varepsilon = 10^{-9}$ added to variances for stability; `predict_proba` uses log-sum-exp trick.
- `TfidfTransformer`: IDF formula $\log \frac{N}{1+\text{df}}$ applied as a broadcast multiply (no loop over documents or vocabulary).

`pledge.txt` (to be updated with signed name and date before submission).

End of Homework 2 report. AI-ENG-201, AI Academy.